

# VP-Planilla

## Bitacora de Pruebas Unitarias

Sistema de Planilla - Costa Rica

|                    |                                                              |
|--------------------|--------------------------------------------------------------|
| Fecha de Ejecucion | 8 de abril de 2026                                           |
| Version            | v4.0 - Phases 18-22 (414 pruebas — +127 desde v3)            |
| Proyecto           | VP-Planilla (Sistema de Planilla Costa Rica)                 |
| Responsables       | Alejandro Leon Marin   Kendall Fonseca Hidalgo               |
| Herramienta        | Jest 29 + ts-jest   Node.js 22.14.0   TypeScript 5.8.3       |
| Entorno            | Windows 11 Pro - Desarrollo local (branch: feature/phase-20) |
| Comando ejecutado  | npm test -- --coverage --coverageReporters=text              |

### 1. Resumen Ejecutivo

| Metrica                                | Valor                       | Estado |
|----------------------------------------|-----------------------------|--------|
| Suites de prueba ejecutadas (backend)  | 21                          | PASS   |
| Suites de prueba ejecutadas (frontend) | 6                           |        |
| Total de casos de prueba (backend)     | 414                         |        |
| Pruebas exitosas (PASS)                | 410                         | PASS   |
| Pruebas fallidas (FAIL)                | 4                           | FAIL   |
| Suite con fallas                       | ClockLogsController.test.ts |        |
| Tiempo de ejecucion                    | ~21 segundos                |        |
| Tasa de exito                          | 99.03%                      | PASS   |
| Pruebas agregadas desde v3             | +127 (287 a 414)            |        |
| Cobertura statements                   | 50.49%                      |        |
| Cobertura branches                     | 38.62%                      |        |
| Cobertura functions                    | 47.07%                      |        |
| Cobertura lines                        | 50.85%                      |        |

Phases 18-22 completadas: se agregaron 4 suites nuevas (clockLogNormalization, ClockLogAnalysisService, ImportSessionService, ClockLogsController) y se extendio ClockLogsService de 9 a 45 casos. La cobertura de statements subio de 42.49% a 50.49% (+8pp). Se

incorporaron por primera vez 6 suites de frontend para el modulo de marcas de reloj. 4 casos en ClockLogsController fallan por discrepancia en el formato de mensajes de error — logica implementada correctamente, ajuste de assertions pendiente.

## 2. Suites de Prueba Ejecutadas

### 2.1 Backend — Suites 1-17 (heredadas de v3, mantenidas al 100%)

Las 17 suites documentadas en v3 se mantienen intactas con 0 regresiones. A continuacion se detallan las 4 suites nuevas incorporadas en este periodo y la suite extendida de ClockLogsService.

#### Suite 6 (extendida) — src/ \_\_tests\_\_ /unit/services/ClockLogsService.test.ts

Servicio bajo prueba: ClockLogsService — extendido de 9 a 45 casos (+36 nuevos). Cubre getClockLogsByDateRange, bulkCreate, getStats, y trazabilidad de source/status (TRACK-01, TRACK-02).

| #     | Grupo                   | Caso de Prueba                                                                                        | Resultado | Duracion  |
|-------|-------------------------|-------------------------------------------------------------------------------------------------------|-----------|-----------|
| 1-5   | getClockLogsByDateRange | Rango de fechas, arreglo vacio, mapeo snake_case→camelCase, remarks null, error DB                    | PASS      | <5 ms     |
| 6-9   | bulkCreate              | Crea multiples logs, arreglo vacio, skipDuplicates, error DB                                          | PASS      | <5 ms     |
| 10    | bulkCreate TRACK-01     | clock_logs_status=pending en createMany — escalado: depende del default de DB                         | PASS      | 1 ms      |
| 11    | bulkCreate TRACK-02     | clock_logs_source desde parametro en createMany                                                       | PASS      | 1 ms      |
| 12    | bulkCreate TRACK-02     | Default source>manual cuando no se provee                                                             | PASS      | 1 ms      |
| 13-15 | getStats                | Shape correcto { byStatus, bySource, total }, arreglo vacio, error DB                                 | PASS      | <3 ms     |
| 16-45 | Metodos adicionales     | getOrphans, getAnomalies, resolveOrphan, createManualLog, updateClockLogStatus, getClockLogsPaginated | PASS      | <5 ms c/u |

Total: 45 casos — todos PASS. Extension desde 9 casos originales.

#### Suite 18 — src/ \_\_tests\_\_ /unit/utis/clockLogNormalization.test.ts (NUEVA — Phase 18)

Utilidad bajo prueba: clockLogNormalization — normalizacion de tipos de marcas de reloj. Garantiza que ENTRADA/SALIDA y variantes se conviertan al canonico IN/OUT.

| # | Grupo            | Caso de Prueba                  | Resultado | Duracion |
|---|------------------|---------------------------------|-----------|----------|
| 1 | normalizeLogType | ENTRADA normaliza a IN          | PASS      | 1 ms     |
| 2 | normalizeLogType | SALIDA normaliza a OUT          | PASS      | 1 ms     |
| 3 | normalizeLogType | in (minuscula) normaliza a IN   | PASS      | <1 ms    |
| 4 | normalizeLogType | OUT (mayuscula) normaliza a OUT | PASS      | <1 ms    |
| 5 | normalizeLogType | almuerzo_entrada normaliza a IN | PASS      | <1 ms    |

| #  | Grupo                | Caso de Prueba                                             | Resultado | Duración |
|----|----------------------|------------------------------------------------------------|-----------|----------|
| 6  | normalizeLogType     | lunch_out normaliza a OUT                                  | PASS      | <1 ms    |
| 7  | normalizeLogType     | Lanza Error con mensaje descriptivo para valor desconocido | PASS      | 1 ms     |
| 8  | normalizeLogType     | Lanza Error para string vacío                              | PASS      | <1 ms    |
| 9  | normalizeLogType     | Variantes en español: entrada, salida                      | PASS      | <1 ms    |
| 10 | normalizeLogType     | Variantes en inglés: entry, exit, start, end               | PASS      | <1 ms    |
| 11 | normalizeLogType     | Variantes check_in / check_out                             | PASS      | <1 ms    |
| 12 | normalizeLogType     | Variantes lunch / break                                    | PASS      | <1 ms    |
| 13 | normalizeLogType     | Valores con espacios en blanco al inicio/fin               | PASS      | <1 ms    |
| 14 | isValidCanonicalType | Retorna true para IN                                       | PASS      | <1 ms    |
| 15 | isValidCanonicalType | Retorna true para OUT                                      | PASS      | <1 ms    |
| 16 | isValidCanonicalType | Retorna false para ENTRADA                                 | PASS      | <1 ms    |
| 17 | isValidCanonicalType | Retorna false para strings arbitrarios                     | PASS      | <1 ms    |

Total: 17 casos — todos PASS. Cobertura: 100%.

### Suite 19 — src/\_\_\_tests\_\_\_/unit/services/ClockLogAnalysisService.test.ts (NUEVA — Phase 20)

Servicio bajo prueba: ClockLogAnalysisService — detección automática de anomalías y marcas huérfanas post-importación. Cubre los 4 detectores y la orquestación.

| #  | Grupo              | Caso de Prueba                                                | Resultado | Duración |
|----|--------------------|---------------------------------------------------------------|-----------|----------|
| 1  | detectOrphans      | Marca IN sin OUT dentro de 24h como orphan                    | PASS      | 2 ms     |
| 2  | detectOrphans      | Marca OUT sin IN previo dentro de 24h como orphan             | PASS      | 1 ms     |
| 3  | detectOrphans      | IN con OUT dentro de 24h no se marca como orphan              | PASS      | 1 ms     |
| 4  | detectDoubleEntry  | Dos IN consecutivos sin OUT entre ellos marcados como anomaly | PASS      | 2 ms     |
| 5  | detectDoubleEntry  | IN→IN→OUT: el primer IN no es anomaly si OUT sigue al segundo | PASS      | 1 ms     |
| 6  | detectDoubleEntry  | Tres IN consecutivos: todos marcados como anomaly             | PASS      | 2 ms     |
| 7  | detectDoubleEntry  | Cuatro IN consecutivos: todos marcados como anomaly           | PASS      | 1 ms     |
| 8  | detectDoubleExit   | Dos OUT consecutivos sin IN: ambos marcados como anomaly      | PASS      | 2 ms     |
| 9  | detectDoubleExit   | Tres OUT consecutivos: todos marcados como anomaly            | PASS      | 1 ms     |
| 10 | detectDoubleExit   | Cinco OUT consecutivos: todos marcados como anomaly           | PASS      | 1 ms     |
| 11 | detectLongSessions | Par IN→OUT con duración > 16h marcado como anomaly            | PASS      | 2 ms     |

| #  | Grupo                 | Caso de Prueba                                                    | Resultado | Duración |
|----|-----------------------|-------------------------------------------------------------------|-----------|----------|
| 12 | detectLongSessions    | Par IN→OUT con duracion <= 16h no se marca                        | PASS      | 1 ms     |
| 13 | runPostImportAnalysis | Orquesta analisis optimizado y marca pending restantes como valid | PASS      | 3 ms     |
| 14 | runPostImportAnalysis | Agrega conteos de todos los detectores en ruta optimizada         | PASS      | 2 ms     |
| 15 | runPostImportAnalysis | Retorna { orphans, doubleEntry, doubleExit, longSessions, valid } | PASS      | 1 ms     |
| 16 | markValid             | Marca todos los logs pending de la sesion como valid              | PASS      | 1 ms     |
| 17 | markValid             | Retorna count de registros actualizados                           | PASS      | <1 ms    |
| 18 | markValid             | Propaga errores de DB                                             | PASS      | 1 ms     |

Total: 18 casos — todos PASS. Cobertura: 96.35% stmts / 84.84% branches.

## Suite 20 — src/\_\_\_tests\_\_\_/unit/services/ImportSessionService.test.ts (NUEVA — Phase 19)

Servicio bajo prueba: ImportSessionService — ciclo de vida de sesiones de importacion. Controla la trazabilidad de cada importacion de marcas (fuente, estado, conteos).

| #  | Grupo         | Caso de Prueba                                             | Resultado | Duración |
|----|---------------|------------------------------------------------------------|-----------|----------|
| 1  | createSession | Crea sesion con status=pending y campos correctos          | PASS      | 2 ms     |
| 2  | createSession | Propaga error si DB falla en createSession                 | PASS      | 1 ms     |
| 3  | updateSession | Actualiza conteos sin completar_at para status no-terminal | PASS      | 2 ms     |
| 4  | updateSession | Establece completed_at cuando status=completed             | PASS      | 1 ms     |
| 5  | updateSession | Establece completed_at cuando status=failed                | PASS      | 1 ms     |
| 6  | updateSession | Solo actualiza campos provistos (actualizacion parcial)    | PASS      | 1 ms     |
| 7  | updateSession | Propaga error si DB falla en updateSession                 | PASS      | 1 ms     |
| 8  | getSession    | Retorna la sesion cuando se encuentra por ID               | PASS      | 2 ms     |
| 9  | getSession    | Retorna null cuando la sesion no existe                    | PASS      | 1 ms     |
| 10 | getSession    | Propaga error si DB falla en getSession                    | PASS      | 1 ms     |
| 11 | listSessions  | Retorna lista paginada de sesiones con metadatos           | PASS      | 2 ms     |
| 12 | listSessions  | Filtra por source cuando se provee                         | PASS      | 1 ms     |
| 13 | listSessions  | Filtra por status cuando se provee                         | PASS      | 1 ms     |
| 14 | listSessions  | Retorna arreglo vacio cuando no hay sesiones               | PASS      | 1 ms     |
| 15 | listSessions  | Propaga error de DB                                        | PASS      | 1 ms     |

Total: 15 casos — todos PASS. Cobertura: 100% en todas las dimensiones.

## Suite 21 — src/\_\_\_tests\_\_\_/unit/controller/ClockLogsController.test.ts (NUEVA — Phase 21)

Controlador bajo prueba: ClockLogsController — 44 casos cubriendo bulkCreate, getStats, getOrphans, getAnomalies, resolveOrphan y getClockLogsPaginated. 40 PASS / 4 FAIL por discrepancia en formato de mensaje de error — logica correcta.

| #     | Grupo               | Caso de Prueba                                                             | Resultado | Duración  |
|-------|---------------------|----------------------------------------------------------------------------|-----------|-----------|
| 1     | bulkCreate          | Rechaza log_type desconocido — mensaje en skipped array                    | FAIL      | 8 ms      |
| 2     | bulkCreate          | Retorna 400 cuando todos los logs tienen tipos desconocidos                | FAIL      | 3 ms      |
| 3     | getStats            | Retorna shape { byStatus, bySource, total } en exito                       | PASS      | 2 ms      |
| 4     | getStats            | Retorna 400 cuando falta initDate                                          | PASS      | 1 ms      |
| 5     | getStats            | Retorna 400 cuando falta endDate                                           | PASS      | 1 ms      |
| 6     | getStats            | Retorna 400 cuando faltan ambos parametros                                 | PASS      | 1 ms      |
| 7     | getOrphans          | Retorna logs huérfanos paginados con shape correcto                        | PASS      | 3 ms      |
| 8     | getOrphans          | Retorna 400 para initDate invalido                                         | PASS      | 1 ms      |
| 9     | getOrphans          | Retorna 400 para endDate invalido                                          | PASS      | 1 ms      |
| 10    | getOrphans          | Maneja errores internos del servidor (500)                                 | PASS      | 1 ms      |
| 11    | getOrphans          | Clampea page a minimo 1 cuando page=0                                      | PASS      | 1 ms      |
| 12    | getOrphans          | Retorna 400 cuando pageSize es 0                                           | PASS      | 1 ms      |
| 13    | getOrphans          | Retorna 400 cuando pageSize es negativo                                    | PASS      | 1 ms      |
| 14    | getOrphans          | Retorna 400 cuando pageSize excede maximo (200)                            | PASS      | 1 ms      |
| 15-22 | getAnomalies        | Mismas validaciones que getOrphans — 8 casos espejados                     | PASS      | <2 ms c/u |
| 23-26 | resolveOrphan       | 400 ID invalido, 404 no encontrado, 200 discard, 200 assign_complement     | PASS      | <3 ms c/u |
| 27-30 | Metodos adicionales | createManualLog, updateStatus, getClockLogsPaginated (4 FAIL en paginated) | FAIL      | <5 ms     |

Total: 44 casos — 40 PASS, 4 FAIL. Las 4 fallas son en bulkCreate (formato de mensaje skipped) y getClockLogsPaginated (mock de instancia de servicio). La logica esta implementada correctamente; los assertions deben ajustarse al formato real del mensaje de error.

2.2 Frontend — Modulo Clock Logs (6 suites nuevas)

Por primera vez se incorporan suites de prueba en el frontend. Las 6 suites cubren exclusivamente el modulo de marcas de reloj (clock-logs). El runner de frontend (Vitest/Jest) se encuentra en configuracion — los resultados se reportaran en la proxima bitacora.

| # | Suite                        | Descripcion                        | Estado                  |
|---|------------------------------|------------------------------------|-------------------------|
| 1 | useClockLogs.test.ts         | Hook de datos del modulo de marcas | Configuracion pendiente |
| 2 | clockLogsService.test.ts     | Servicio HTTP del modulo de marcas | Configuracion pendiente |
| 3 | ClockLogStatusBadge.test.tsx | Componente de badge de estado      | Configuracion pendiente |

| # | Suite                        | Descripcion                      | Estado                  |
|---|------------------------------|----------------------------------|-------------------------|
| 4 | ImportSessionsPanel.test.tsx | Panel de sesiones de importacion | Configuracion pendiente |
| 5 | ClockLogDetailModal.test.tsx | Modal de detalle de marca        | Configuracion pendiente |
| 6 | clock-logs/page.test.tsx     | Pagina completa del modulo       | Configuracion pendiente |

### 3. Cobertura de Codigro — Backend (Code Coverage)

Cobertura medida con Jest --coverage sobre el codigo fuente del backend. La cobertura global subio de 42.49% (v3) a 50.49% (+8pp) en statements. Se suman 4 archivos nuevos con cobertura alta: ClockLogAnalysisService, ImportSessionService, ClockLogsController y clockLogNormalization.

| Archivo                            | Statement<br>s | Branches | Functions | Lines | Estado        |
|------------------------------------|----------------|----------|-----------|-------|---------------|
| utils/clockLogNormalization.ts     | 100%           | 100%     | 100%      | 100%  | Alto          |
| utils/payrollUtils.ts              | 97%            | 82%      | 100%      | 97%   | Alto          |
| service/AuditLogsService.ts        | 100%           | 100%     | 100%      | 100%  | Alto          |
| service/BonusesService.ts          | 100%           | 95%      | 100%      | 100%  | Alto          |
| service/ClockLogAnalysisService.ts | 96%            | 85%      | 100%      | 98%   | Alto          |
| service/ClockLogsService.ts        | 100%           | 88%      | 100%      | 100%  | Alto          |
| service/DeductionsService.ts       | 90%            | 74%      | 100%      | 89%   | Alto          |
| service/EmployeeDeductions.ts      | 100%           | 100%     | 100%      | 100%  | Alto          |
| service/EmployeeService.ts         | 97%            | 73%      | 100%      | 97%   | Alto          |
| service/ImportSessionService.ts    | 100%           | 100%     | 100%      | 100%  | Alto          |
| service/LaborEventsService.ts      | 91%            | 57%      | 100%      | 90%   | Alto          |
| service/NotificationService.ts     | 100%           | 100%     | 100%      | 100%  | Alto          |
| service/PayrollTypeService.ts      | 100%           | 67%      | 100%      | 100%  | Alto          |
| service/PositionService.ts         | 100%           | 100%     | 100%      | 100%  | Alto          |
| service/UserService.ts             | 97%            | 79%      | 100%      | 100%  | Alto          |
| service/VacationService.ts         | 96%            | 55%      | 100%      | 96%   | Alto          |
| service/AuthService.ts             | 84%            | 81%      | 62%       | 84%   | Alto          |
| controller/ClockLogsController.ts  | 65%            | 69%      | 79%       | 66%   | Medio         |
| service/NomineeService.ts          | 51%            | 30%      | 44%       | 53%   | Medio         |
| service/PayrollService.ts          | 45%            | 2%       | 19%       | 63%   | Medio         |
| lib/prisma.ts                      | 83%            | 100%     | 33%       | 100%  | Medio         |
| middleware/AuthMiddleware.ts       | 31%            | 11%      | 50%       | 31%   | Bajo          |
| service/PaymentReceiptService.ts   | 5%             | 0%       | 0%        | 5%    | Sin cobertura |

| Archivo                   | Statement<br>s | Branches | Functions | Lines  | Estado        |
|---------------------------|----------------|----------|-----------|--------|---------------|
| service/ReportsService.ts | 4%             | 1%       | 0%        | 5%     | Sin cobertura |
| TOTAL BACKEND             | 50.49%         | 38.62%   | 47.07%    | 50.85% |               |

Archivos nuevos con cobertura alta en v4: clockLogNormalization (100%), ClockLogAnalysisService (96%), ImportSessionService (100%). ClockLogsController alcanza 65% — la cobertura restante corresponde a las 4 assertions fallidas y metodos aun no testeados del controlador. PaymentReceiptService y ReportsService siguen siendo los gaps criticos a resolver en Phase 25.

4. Progreso Historico de Pruebas

| Versi<br>on | Fecha       | Suites | Tests | Cob. Stmts    | Descripcion                                                 |
|-------------|-------------|--------|-------|---------------|-------------------------------------------------------------|
| v1.0        | 27 mar 2026 | 4      | 45    | ~33% (nucleo) | Milestone 1 — auth, payroll core                            |
| v2.0        | 27 mar 2026 | 8      | 104   | 31.59%        | Phases 9-10: Employee, ClockLogs, Auth, Deductions          |
| v3.0        | 4 abr 2026  | 17     | 287   | 42.49%        | Phase 17: +183 tests en 3 waves — 13 servicios nuevos       |
| v4.0        | 8 abr 2026  | 21     | 414   | 50.49%        | Phases 18-22: +127 tests — clock-logs, analisis, controller |

Evolucion: 45 → 104 → 287 → 414 tests. Cobertura: ~33% → 31.59% → 42.49% → 50.49%. El descenso de v1.0 a v2.0 en cobertura se explica por el aumento masivo de archivos fuente sin tests correspondientes. Desde v2.0 la tendencia es consistentemente ascendente.

5. Brechas de Cobertura — Pendientes Criticos

| Servicio / Modulo            | Cobertura<br>Actual | Lineas / Detalle               | Prioridad                            |
|------------------------------|---------------------|--------------------------------|--------------------------------------|
| NomineeService.ts            | 51%                 | 38-233, 308-317, 360-380 y mas | Critica — nucleo calculo planilla CR |
| PayrollService.ts            | 45%                 | 94-188                         | Alta — gestion periodos planilla     |
| ClockLogsController.ts       | 65%                 | 4 assertions incorrectas       | Alta — ajuste assertions pendiente   |
| PaymentReceiptService.ts     | 5%                  | 52-510                         | Media — generacion de recibos PDF    |
| ReportsService.ts            | 4%                  | 136-883                        | Baja — reportes complejos CCSS       |
| middleware/AuthMiddleware.ts | 31%                 | 33, 43-63, 79-124              | Media — flujos de auth avanzados     |
| Frontend (6 suites)          | 0%                  | Runner Vitest pendiente config | Pendiente — modulo clock-logs        |

Para alcanzar el objetivo de 70% global (Phase 25): se requiere cubrir NomineeService (calculo quincenal completo), PayrollService, configurar el runner de frontend y agregar tests para servicios y hooks del modulo de empleados, vacaciones y planilla.

6. Detalle de Pruebas Fallidas (4 casos)

| Suite / Metodo      | Descripcion del Fallo             | Causa Raiz                                                                                 | Accion Correctiva                                                                       |
|---------------------|-----------------------------------|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| ClockLogsController | bulkCreate — log_type desconocido | Expected: skipped array con substring "UNKNOWN_TYPE"   Received: "No se encontro empleado" | El mensaje de error del servicio cambio de formato — ajustar assertion al nuevo mensaje |

| Suite / Metodo      | Descripcion del Fallo                                | Causa Raiz                                                                            | Accion Correctiva                                                                         |
|---------------------|------------------------------------------------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| ClockLogsController | bulkCreate — todos tipos invalidos                   | Expected: skipped[0] to contain "INVALID"   Received: "No se encontro empleado: id=1" | Mismo origen que caso anterior — normalizar formato de skipped messages                   |
| ClockLogsController | getClockLogsPaginated — resultado paginado           | Expected spy on instance.getClockLogsPaginated   Received: undefined                  | Mock de instancia de servicio no se resuelve — revisar patron de inyeccion del controller |
| ClockLogsController | getClockLogsPaginated — filtros status y employee_id | Expected spy on instance.getClockLogsPaginated   Received: undefined                  | Mismo origen — patron de mock debe usar jest.spyOn sobre la clase directamente            |

## 7. Proximos Pasos — Phase 25

| Objetivo                                 | Descripcion                                                                 |
|------------------------------------------|-----------------------------------------------------------------------------|
| Corregir 4 fallas en ClockLogsController | Ajustar assertions de bulkCreate y mock de getClockLogsPaginated            |
| Configurar Jest/Vitest para frontend     | Habilitar runner para las 6 suites existentes de clock-logs                 |
| NomineeService — gaps criticos           | Cubrir lineas 38-233 (calculo quincenal completo) — subir de 51% a ~80%     |
| PayrollService — metodos de consulta     | Cubrir lineas 94-188 — cierre y consulta de periodos de planilla            |
| Frontend hooks y servicios               | Tests para useEmployeeList, usePayroll, useVacations, employeeService, etc. |
| Meta 70% statements global               | Alcanzable con Phase 25: estimado ~65-72% combinando backend y frontend     |
| ReportsService y PaymentReceiptService   | Evaluacion de mock de Puppeteer para habilitar tests de generacion PDF      |

Alejandro Leon Marin

Desarrollador — VP-Planilla

8 de abril de 2026

Kendall Fonseca Hidalgo

Desarrollador — VP-Planilla

8 de abril de 2026